Documento de alcance del proyecto

AI que te pillo · Grupo L · Seguridad en Entornos Móviles, curso 2026/27 · Bloque II
Versión 1, borrador del equipo · 5 de noviembre de 2026

Miembros: José Ignacio Aranda Gómez-Guillamón, Jorge Varea Durán, Alejandro Galán Rita y Gonzalo Ramos Tarifa.
Repositorio: https://github.com/Ignacio-Aranda/ai_que_te_pillo

1. La aplicación

AI que te pillo es un juego para grupos de amigos que pasan juntos unas horas o unos días: una casa rural, un viaje, una clase o una fiesta. Cada jugador recibe misiones generadas por inteligencia artificial y adaptadas al contexto del grupo, del tipo «pilla a Ignacio bebiendo agua». Para completar una misión hace una foto con la cámara de la aplicación, y una IA comprueba que la foto la cumple comparándola con el selfie que el objetivo se hizo al registrarse. Cada misión vale 1, 3 o 5 puntos según su dificultad, y los puntos forman la clasificación del grupo.

La usarían grupos de amigos ya formados. Cada grupo tiene un código de invitación de seis caracteres; para unirse hay que escribirlo o abrir el enlace `aiquetepillo://unirse/{codigo}` que comparte otro miembro. El dato más sensible son las fotos de personas: el selfie de registro es un dato biométrico, categoría especial según el RGPD, y las fotos de las misiones pueden mostrar a amigos en situaciones comprometidas. Unas y otras se suben al servidor y se envían a un proveedor de IA externo.

Relación con el dominio asignado. Al grupo L le correspondió *Enlace: mensajería local entre dispositivos*. La aplicación conserva lo que hace delicado ese dominio: conecta los dispositivos de personas que están juntas y hace circular entre ellos contenido sobre ellas. El permiso natural del dominio (contactos o proximidad) se sustituye por la cámara, que es la que exige la función principal; la proximidad queda como mejora opcional para comprobar que cazador y objetivo están cerca.

Estado al cierre del Bloque II. La aplicación parte del proyecto base de la asignatura y cubre el juego completo: registro con un selfie hecho con la cámara frontal, acceso contra el backend con la sesión guardada en `SharedPreferences`, grupos con código e invitación por enlace, misiones generadas por el modelo `anthropic/claude-haiku-5.5` a través de OpenRouter, foto de la misión con CameraX, verificación de la foto por el mismo modelo y clasificación por grupo. Sin clave del proveedor, el backend usa misiones de una lista fija y acepta todas las fotos. Es insegura a propósito (datos en claro, registros en logcat y HTTP), y cada punto se corrige en el Bloque III.

2. Datos que maneja

Inventario iniciado en P2.2, con el estado del código al cierre del Bloque II. La última columna se completa en el Bloque III.

Dato	Dónde se guarda hoy	Quién puede leerlo hoy	Cómo se protegerá
Token de sesión	`shared_prefs/sesion.xml`, en claro. También en logcat (el `Log.d` de la respuesta de acceso y el interceptor de OkHttp) y en la cabecera de cada petición HTTP	Cualquiera con adb y la depuración activada; cualquiera en la misma red; la copia de seguridad en la nube. Además es predecible: Base64 de `usuario:hora`	Bloque III
Contraseña	No se guarda en el móvil. Viaja en claro al iniciar sesión y queda en logcat. En el servidor está en claro	Quien lea logcat; cualquiera en la misma red; quien acceda al servidor	Bloque III
Nombre de usuario y nombre visible	`sesion.xml` y la tabla `misiones` de Room, en claro	Cualquiera con adb y la depuración activada; la copia de seguridad	Bloque III
Misiones (título, descripción, puntos, fecha)	`databases/aiquetepillo.db` con sus ficheros `-wal` y `-shm`, sin cifrar, y en logcat. Las descripciones nombran a personas reales	Cualquiera con adb y la depuración activada; la copia de seguridad; la red. El servidor entrega la misión de otro usuario a quien pida su número	Bloque III
Perfil (nombre, correo, DNI, rol)	No se guarda en el móvil. En el servidor, en claro	Cualquier usuario con sesión puede listar los de todos: `/api/admin/usuarios` no comprueba el rol	Bloque III
Selfie de registro	`cache/fotos/selfie.jpg`, sin cifrar, hasta que se sube con el registro; después se borra. Viaja en Base64 por HTTP y queda en logcat. En el servidor, en memoria y sin cifrar. Es un dato biométrico (art. 9 del RGPD)	Cualquiera con adb o en la misma red durante el registro; quien acceda al servidor; el proveedor de IA, que lo recibe en cada comprobación en la que su dueño es el objetivo	Bloque III
Fotos de las misiones	`cache/fotos/mision_<id>.jpg`, sin cifrar; se borran al conseguir o saltar la misión y al cerrar sesión. Se envían en Base64 por HTTP y quedan en logcat. El servidor no las guarda: las reenvía al proveedor de IA	Cualquiera con adb y la depuración activada; la red; un tercero, el proveedor de IA	Bloque III
Contexto y código de invitación del grupo	En el servidor, en memoria y en claro. En el móvil no se guardan, pero viajan por HTTP y quedan en logcat. El código va también en el texto de la invitación	Los miembros y quien reciba la invitación; adb y la red; quien adivine el código, porque la API no limita los intentos	Bloque III
Clave del proveedor de IA	Solo en el servidor, en `backend/.env` (excluido del repositorio) o en una variable de entorno. Nunca en el APK	Quien acceda al servidor. Si llegase al APK, cualquiera que lo decompile	Bloque III

3. Superficie expuesta

Inventario de la sección 5 de P2.3 sobre el paquete `es.uma.aiquetepillo`. En nuestro manifiesto solo declaramos exportada `MainActivity`, pero el APK de depuración exporta además dos actividades que añaden las bibliotecas de Compose y un receptor de AndroidX protegido con `DUMP`, que solo pueden usar el sistema y `adb`.

Vía de entrada	Cómo se invoca hoy	Qué comprueba hoy	Se trata en
Activity principal exportada	`adb shell am start -n es.uma.aiquetepillo/.MainActivity`, o un `Intent` desde otra app	Nada	Sesión 3.4

Vía de entrada	Cómo se invoca hoy	Qué comprueba hoy	Se trata en
Actividades de Compose exportadas (solo en depuración)	`adb shell am start -n es.uma.aiquetepillo/androidx.compose.ui.tooling.PreviewActivity`, o igual con `androidx.activity.ComponentActivity`	Nada: `ui-tooling` y `ui-test-manifest` las exportan sin permiso. `PreviewActivity` dibuja el *composable* que se le indique, sin pasar por la navegación	Sesión 3.4
Enlace profundo `aiquetepillo://mision/{id}`	`adb shell am start -a android.intent.action.VIEW -d "aiquetepillo://mision/3" es.uma.aiquetepillo`, o un enlace en un mensaje o una web	Nada en la interfaz: se salta el acceso y solo exige que el id sea un número. Con un token guardado muestra también misiones ajenas	Sesión 3.4
Llamadas a la API	`curl` con un token válido	Solo la presencia del token. No comprueba de quién es la misión ni el rol de administrador, y el acceso revela si un usuario existe	Sesión 3.5 · P3.2 (canal) · Bloque IV (autorización)
Canal de red	Cualquiera en la misma red o con un proxy	Nada: HTTP en claro, permitido por `usesCleartextTraffic`	Sesión 3.5 · P3.2
Almacenamiento local	`adb shell run-as es.uma.aiquetepillo`	Nada: está en claro	Sesión 3.3 · P3.1
Copia de seguridad	Copia automática en la nube de Google	Nada: `allowBackup="true"` con reglas vacías	Sesión 3.3 · P3.1
Registros	`adb logcat` filtrando por `Repositorio` y `okhttp`	Nada: salen el usuario, la contraseña, el token y las respuestas	Sesión 3.6
Vista de recientes y capturas	Botón de recientes o captura de pantalla	Protegida con `FLAG_SECURE`	Sesión 3.6
Permiso de cámara	Botones «HACER SELFIE» del registro y «HACER FOTO» de la misión	Se pide en tiempo de ejecución y la app funciona si se deniega. La cámara (frontal para el selfie, trasera para las misiones) va con CameraX, comprueba el permiso al abrirse y no tiene enlace profundo	Bloque III · Bloque IV
Enlace `aiquetepillo://unirse/{codigo}`	`adb shell am start -a android.intent.action.VIEW -d "aiquetepillo://unirse/K7P2QX"`, o el texto que comparte «INVITAR»	Enseña el grupo y solo une al pulsar «UNIRSE», con el token guardado. El código se puede probar por fuerza bruta: sin límite de intentos y con un 404 para los que no existen	Sesión 3.4 · Bloque IV
Subida de fotos y envío al proveedor de IA	«ENVIAR FOTO», o `curl` contra `POST /api/misiones/{id}/foto`	Que haya token, que la misión sea del usuario y que sea un JPEG. No limita el tamaño ni los intentos, y el modelo recibe el contexto del grupo y la foto, que pueden llevar instrucciones inyectadas	Sesión 3.5 · Bloque IV

4. Modelo de amenazas preliminar

Quién atacaría la aplicación y qué buscaría: escenarios de partida para el modelo de amenazas del Bloque III.

1. **Alguien que se cuela en un grupo.** Un conocido al que le reenvían el enlace de invitación, o alguien que prueba códigos al azar, entra en un grupo ajeno. Busca ver las fotos y los selfies de los miembros y saber dónde están y qué hacen. Si el código es corto y no caduca, le basta con probar o con recibir el enlace.
2. **Alguien con el móvil en la mano o en la misma wifi.** Un amigo que coge el teléfono desbloqueado y lo conecta a su portátil, o cualquiera en la wifi compartida de la casa rural. Busca hacerse pasar por el dueño: jugar en su nombre o leer lo que guarda la aplicación. Hoy lo tiene fácil: la sesión, las misiones, las fotos y la contraseña quedan en claro en el móvil, en los registros y en la red.
3. **Alguien que busca las fotos para difundirlas.** Las fotos de las misiones muestran a personas en situaciones que no querrían ver publicadas, y el selfie sirve para reconocer o suplantar una cara. Quien entre en el servidor, intercepte la subida o un proveedor de IA que conserve las imágenes podría usarlas para humillar, chantajear o suplantar, y desde el servidor se llevaría además la clave del proveedor para gastarla a costa del proyecto. Es el daño más grave, porque alcanza a personas que solo han salido en una foto.
4. **Un jugador tramposo.** Un miembro que quiere ganar sin cumplir las misiones: sube una foto antigua, retocada o de alguien parecido para engañar a la IA, escribe en la foto una orden para que el modelo la dé por buena, o le pide al servidor misiones que no son suyas. Busca puntos, y de paso ve las misiones de los demás: hoy el servidor entrega la misión de cualquier usuario a quien pida su número.

5. Reparto de trabajo

Cada miembro responde de un requisito de principio a fin: lo construye (Bloque II), lo protege (Bloque III) y lo defiende en la auditoría (Bloque IV). La API remota es lo más grande, así que su responsable no lleva tareas extra.

Miembro	Requisito técnico mínimo	Además	Lo protege en
José Ignacio Aranda Gómez-Guillamón	Llamada a una API remota: backend de grupos, misiones y clasificación; generación y verificación con el proveedor de IA	—	P3.2 (TLS y anclaje de certificados)
Jorge Varea Durán	Autenticación de usuarios: registro e inicio de sesión en app y backend, gestión del token y cierre de sesión	Coordinar este documento	Sesión 3.5 · Bloque IV (tokens y autorización)
Alejandro Galán Rita	Almacenamiento local de datos sensibles: sesión, caché de Room de grupos y misiones, borrado al cerrar sesión	Pantalla de clasificación	P3.1 (cifrado en reposo)
Gonzalo Ramos Tarifa	Permiso peligroso justificado: cámara con CameraX para el selfie de registro y las fotos de las misiones	Enlace de invitación y su pantalla de confirmación	P3.3 (componentes, permisos y fugas)